

JAVA PROGRAMMING PRACTICAL RECORD

Longest Substring Without Repeating Characters

Experiment	Longest Substring Without Repeating Characters
Technique	Sliding Window
Language	Java

1. Aim

To write a Java program to find the longest substring without repeating characters using the sliding window technique and HashSet.

2. Problem Statement

Given a string, find the longest substring that contains no repeated characters. The substring must contain contiguous characters.

Example: abcabcbb → Length = 3, String = abc

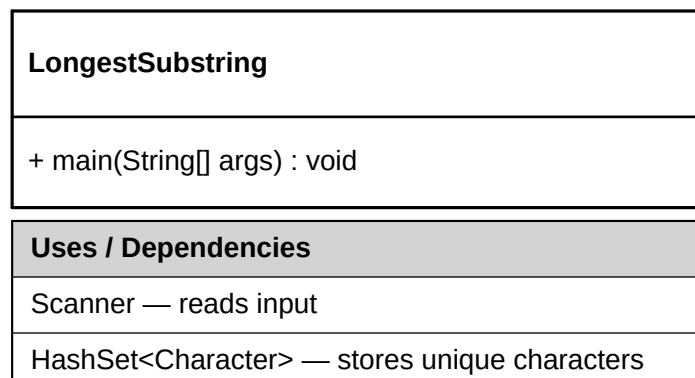
Example: pwwkew → Length = 3, String = wke

Note: pwke is not a substring because its characters are not all contiguous; it is a subsequence.

3. Algorithm

1. Read the input string.
2. Create a HashSet to store characters in the current window.
3. Use two pointers, left and right, to represent the window.
4. Move the right pointer through the string.
5. If the current character is already present, remove characters from the left until the duplicate is removed.
6. Add the current character to the HashSet.
7. Calculate the current window length.
8. If the current length is greater than the maximum length, store its length and starting position.
9. Extract the longest substring using substring().
10. Display the substring and its length.

4. Class UML Diagram



5. Program

```
import java.util.HashSet;
import java.util.Set;
import java.util.Scanner;

public class LongestSubstring {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        Set<Character> set = new HashSet<>();

        int left = 0;
        int maxLength = 0;
        int start = 0;

        for (int right = 0; right < str.length(); right++) {

            while (set.contains(str.charAt(right))) {
                set.remove(str.charAt(left));
                left++;
            }

            set.add(str.charAt(right));

            if (right - left + 1 > maxLength) {
                maxLength = right - left + 1;
                start = left;
            }
        }

        String longest = str.substring(start, start + maxLength);

        System.out.println("Longest substring: " + longest);
        System.out.println("Length: " + maxLength);

        sc.close();
    }
}
```

6. Sample Output – 1

```
Enter a string: abcabcbb
Longest substring: abc
Length: 3
```

7. Sample Output – 2

```
Enter a string: pwwkew
Longest substring: wke
Length: 3
```

8. Explanation

The program maintains a sliding window containing only unique characters. The left pointer moves forward whenever a duplicate character is found. The HashSet provides efficient checking, insertion, and removal of characters. The start position and maximum length are updated whenever a longer valid window is found.

9. Result

Thus, the Java program successfully finds and displays the longest substring without repeating characters along with its length.